Greenhouse Management

Analysis and Advice

by Kaloyan Milev

One plant prototype | Four week project

Table of Contents

1. Introduction....2
2. Goals and Objectives....2
    2.1 Main goal....2
    2.2 SMART goal....2
    2.3 Learning objectives....2
3. Scope....2
    3.1 In scope....2
    3.2 Out of scope....3
4. Requirements....3
5. Approach....4
5. Target Audience....4
6. Advice....4
7. Technical Details....5

1. Introduction

This project involves building a greenhouse management prototype for one plant within four weeks. It monitors growing conditions, controls irrigation locally and presents information on a local display and a remote dashboard. Measurements and watering events are stored so the caretaker can review what happened over time.

Watering decisions are made at the plant, while a separate computer stores records and provides the dashboard and alerts. This allows local plant care to continue during a network failure. Prototype hardware is sufficient for this project; commercial deployment is outside the scope.

2. Goals and Objectives

2.1 Main goal

Build and test an integrated one-plant prototype with environmental monitoring, local automatic irrigation, a local display, database storage, a dashboard and alerts within the four-week project period.

2.2 SMART goal

SMART criterion	Application to the greenhouse project
Specific	Monitor one plant and one reservoir, automate watering, and show soil moisture, water availability, temperature and humidity locally and remotely. Add light monitoring if time allows.
Measurable	The prototype completes a plant-care cycle: detect dry soil, water when enough reservoir water is available, stop at the watering limit, and record the activity. Low water or an unreliable sensor reading prevents watering. The caretaker can view current and past readings and recognise alerts. Local monitoring and watering continue during a network failure.
Achievable	Use available hardware and my embedded programming experience. Complete the core functions before adding light monitoring, a fan or remote settings.
Relevant	Improve my embedded skills and my ability to document, design and realise a professional IT product.
Time bound	Complete and test the prototype within the four-week project period.

2.3 Learning objectives

- Improve my embedded C programming and learn the required frameworks.
- Connect an embedded device to messaging, persistent storage and a dashboard.
- Use datasheets and official documentation to explain technical decisions independently.
- Improve my process of analysing, designing, building and testing an integrated system.

3. Scope

3.1 In scope

- One plant, one watering zone and one external water reservoir.

- Monitoring soil moisture, air temperature, humidity, light, and reservoir water availability.
- Automatic watering and a local display of readings and system status.
- Stored readings and watering records, a remote dashboard and alerts.
- Light monitoring if time allows; fan control and remote settings as optional additions.

3.2 Out of scope

- Automatic heating, humidification, dehumidification and artificial lighting.
- Plant disease detection, weather forecasting and AI control.
- Multiple independent growing zones.
- Commercial-scale deployment and industrial-grade hardware.

4. Requirements

Functional requirements (FR) use MoSCoW priorities: Must Have means essential, Should Have means desirable, Could Have means optional, and Won't Have means excluded from this project.

ID	Requirement	Priority
FR-01	Measure soil moisture, air temperature and humidity.	Must Have
FR-02	Detect when the reservoir has too little water for watering.	Must Have
FR-03	Water dry soil automatically when enough water is available, and stop when the set watering limit is reached.	Must Have
FR-04	Keep the pump off when water is too low or the readings needed for watering are missing. Keep it off until valid readings are available after startup.	Must Have
FR-05	Show current readings, watering status and warnings on a local display.	Must Have
FR-06	Send measurements and watering events to the computer that runs the dashboard and stores the data.	Must Have
FR-07	Save measurements and watering events in a database on the computer running the dashboard so they can be reviewed later.	Must Have
FR-08	Show current readings, historical trends and watering activity on a remote dashboard.	Must Have
FR-09	Alert the caretaker about low water, temperature or humidity outside the chosen limits, and detected system faults.	Must Have
FR-10	Continue local monitoring and automatic watering during a network failure, and resume sending data when the connection returns.	Must Have
FR-11	Measure light levels, save and display the readings, and warn when they are outside the chosen range.	Should Have
FR-12	Control a ventilation fan automatically using temperature and/or humidity.	Could Have
FR-13	Allow the user to change selected settings remotely, such as watering or alert thresholds.	Could Have
FR-14	Control heating automatically.	Won't Have

ID	Requirement	Priority
FR-15	Control air humidity automatically.	Won’t Have
FR-16	Control artificial lighting automatically.	Won’t Have
FR-17	Detect plant diseases.	Won’t Have
FR-18	Forecast the weather.	Won’t Have
FR-19	Use AI or machine learning to control growing conditions.	Won’t Have
FR-20	Manage multiple independent growing zones.	Won’t Have
FR-21	Support commercial-scale use with industrial-grade hardware.	Won’t Have

5. Approach

I will use a Waterfall model to organise the project into clear phases: analysis and advice, design, realization, and testing. This suits the defined scope of a one-plant prototype and the four-week project period.

First, I will define the requirements and compare suitable technologies. Next, I will design the hardware connections, software structure and communication between components. I will then build the core monitoring and watering functions before adding database storage and the dashboard. Optional features will follow only if time allows.

I will check individual components during implementation and test the complete system against the requirements once it is integrated. If a hardware limitation or test result requires a change, I will update the affected requirement or design before continuing.

The schedule for these phases is shown in the Project Plan.

5. Target Audience

The primary user is the caretaker of the plant. The caretaker needs to recognise whether conditions are acceptable, whether watering occurred and when refilling or inspection is needed. The prototype should reduce routine watering work while leaving reservoir refilling and responses to environmental alerts with the caretaker.

A technical maintainer supports the device and its software, sensor calibration, accounts, backups and recovery. These responsibilities must be assigned at handover.

6. Advice

Start with one plant and focus on reliable watering and monitoring before adding optional features. Keep watering control at the plant so it continues when the network or dashboard is unavailable.

Reuse the available sensors and pump if they are suitable for the chosen plant and reservoir. Set watering and warning limits for that plant. Use an existing suitable computer for data storage and the dashboard before buying a dedicated one.

Assign someone to refill the reservoir, inspect the pump and tubing, and respond to warnings. Temperature and humidity alerts require the caretaker to act because the core system does not correct these conditions automatically.

7. Technical Details

Hardware and software comparisons, and the reasons for choosing each part and technology, are covered in the separate Greenhouse Technology Selection document:

Greenhouse_Technical_Details_and_Technology_Selection.docx